

CINEMATCH

AI + C Engine · Mood-Based Movie & Game Recommender

Advanced C Programming · Sejong University · 2024

Member 1 [Name]

Member 2 [Name]

Member 3 [Name]

Member 4 [Name]

Member 5 [Name]

Member 6 [Name]

The Problem

"You've been scrolling for 20 minutes and still can't decide what to watch."



Too Many Choices

Netflix has 15,000+ titles.
Steam has 50,000+ games.
More options = harder decisions.



No Mood Filter

Platforms use viewing history —
not how you feel tonight.
Past ≠ Present.



Context is Ignored

Are you alone or with friends?
Tired or energetic?
Existing systems don't ask.

CINEMATCH — The Solution

You type:

"something scary but short to watch with friends tonight"

1

AI reads your mood

Converts natural language
into 5 structured preferences
(genre, time, mood, medium, social)

2

C engine scores

Runs a weighted algorithm
across 55 curated titles
and picks the best match

3

AI explains

Writes a personalized
explanation of why that
specific title fits you now

5 Features of CINEMATCH



AI Match

Describe your mood → AI + C engine recommends the perfect title with a personalized explanation



Movies Browser

Search & browse hundreds of thousands of real films via TMDB. Posters, cast, full details.



Games Browser

Search & browse 800,000+ titles via RAWG game database. Genre filters, cover art.



Discussion

Pick any movie → AI generates theme analysis, epic moments, director's vision, discussion Qs.



History & Accounts

Register, log in, and see all your past AI picks saved automatically with date & mood.

What Makes Each Feature Special

🌟 AI Match — Core Feature

The only recommender that converts your current mood into structured preferences and scores them with compiled C code. Result in under 10 seconds.

📄 History — Automatic Saving

Every AI recommendation is silently saved the moment it appears. Reload the page next week — your history is still there.

🎬 Movies + 🎮 Games Browser

Real open databases — TMDB and RAWG. Not a static list. Search anything, filter by 8 genres, see posters and cover art.

🔒 User Accounts

Register with a username and password. Passwords are hashed with bcrypt — never stored in plain text. JWT sessions last 7 days.

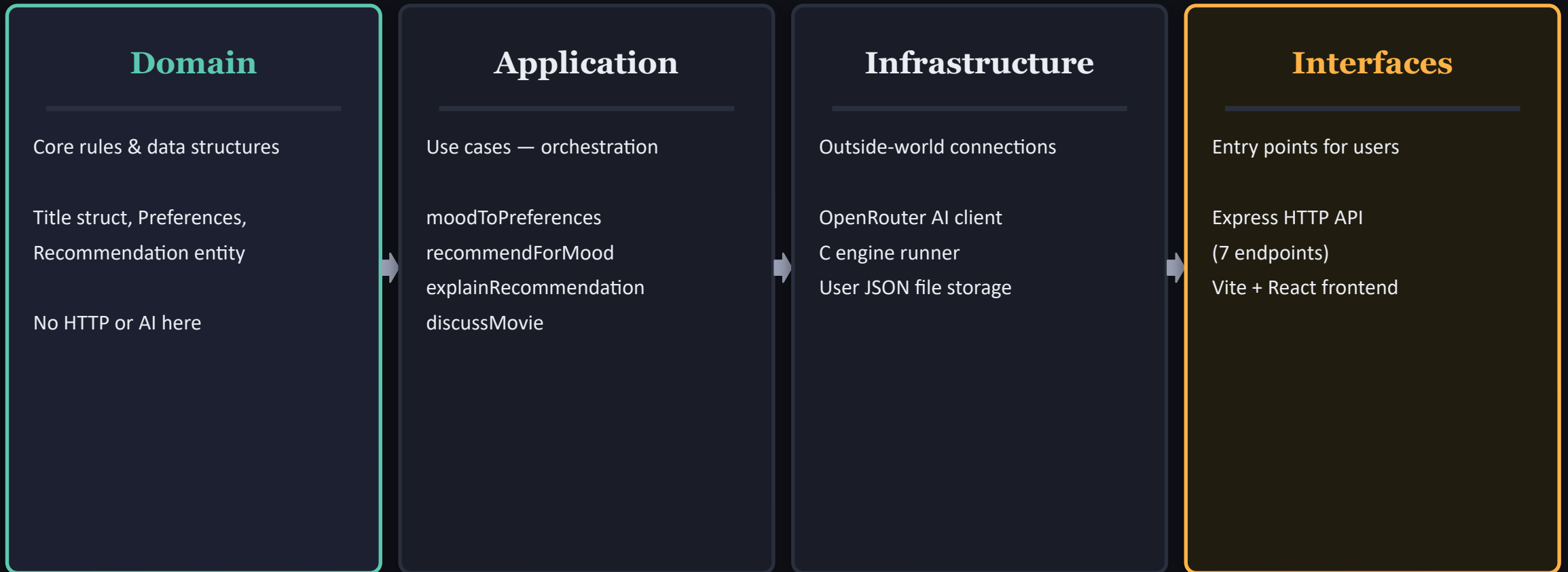
💬 Discussion

Powered by AI + TMDB cast data. Get main theme, epic moments, director's vision, and ready-made discussion questions for any movie.

🛡️ API Key Security

TMDB, RAWG, and OpenRouter keys live in .env files that are gitignored. Never committed. Never exposed in the browser.

System Architecture — Domain-Driven Design



🔧 C Engine (cinematch.exe) — compiled binary called by Infrastructure layer via `child_process.spawn()`

C Engine — Data Structure & Scoring Algorithm



```
/* titles.h – the Title data structure */
typedef struct {
    const char *type;    /* "Movie" or "Game"    */
    const char *title;   /* display name      */
    int        year;
    const char *genre;   /* action/cozy/scifi... */
    const char *mood;    /* intense/relaxed/deep */
    const char *timeNeed; /* short/medium/long    */
    const char *social;  /* solo/group/any       */
    const char *rating;  /* e.g. "8.6"           */
    const char *desc;
```



```
/* Scoring weights in recommend.c */
#define PTS_GENRE      6 /* strongest signal */
#define PTS_MEDIUM    5
#define PTS_MOOD       4
#define PTS_TIME_EXACT 3
#define PTS_TIME_ADJ   1 /* neighbouring length */
#define PTS_SOCIAL     2
/* rating / 10.0 added as tiebreaker */
```

Scoring Rules

Criterion	Points	Why
Genre match	6	Strongest preference signal
Medium match	5	Movie vs Game is binary
Mood match	4	Emotional tone matters most
Time — exact	3	Short/medium/long fit
Time — adjacent	1	Close enough still counts
Social match	2	Solo vs group is clear
Rating ÷ 10	tiebreaker	E.g. 8.6 → adds +0.86

C Engine — Engine Mode & Node.js Integration

```
/* main.c — two modes */
int main(int argc, char **argv) {

    /* Engine mode: Node calls with 5 args */
    if (argc == 6) {
        recommendJson(argv[1], argv[2],
                      argv[3], argv[4], argv[5]);
        return 0; /* prints JSON, exits */
    }

    /* Interactive mode: asks 5 questions */
    /* ... quiz code ... */
}
```

```
/* Output printed to stdout (Node reads this): */
{"found":true,"type":"Movie",
 "title":"Alien","year":1979,
 "genre":"horror","rating":"8.5"}
```

```
// infrastructure/cEngine.js
const { spawnSync } = require('child_process');

function callEngine(prefs) {
    const r = spawnSync('./cinematch.exe',
        [prefs.medium, prefs.genre,
         prefs.timeNeed, prefs.mood,
         prefs.social]);

    return JSON.parse(r.stdout.toString());
}
```

Why this design?

- C and JavaScript stay completely independent
- JSON is the language-agnostic contract
- C runs as a separate process — maximum speed
- Same C binary works in terminal and from the server

AI Integration — OpenRouter & Fallback Chain

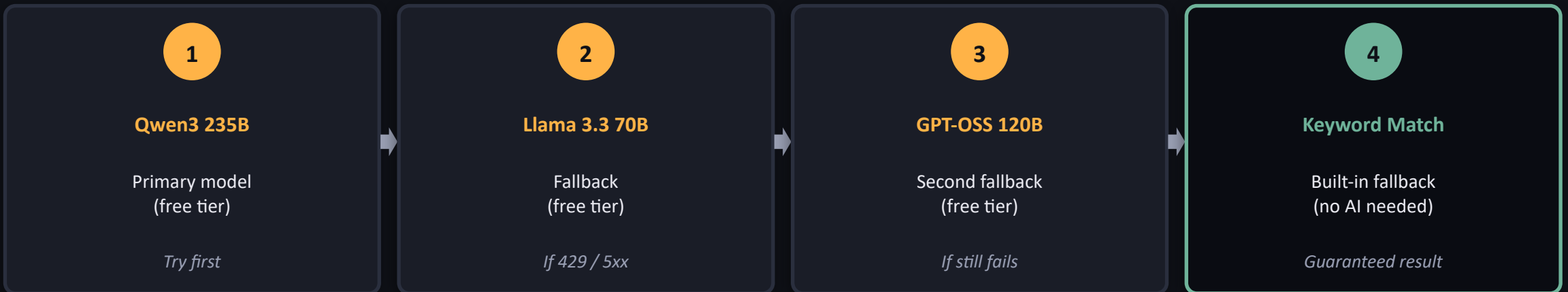
What is OpenRouter?

A single API endpoint that gives access to dozens of large language models — the same interface as OpenAI.
We use free-tier models, so running CINEMATCH costs \$0.

Why a fallback chain?

Free AI models have rate limits. During a live demo, any single model might be rate-limited. The chain guarantees a result is always returned — even without any AI.

Fallback Chain (runs automatically on rate-limit or server error)



AI Pipeline — Mood → Preferences → Explanation

Task 1 — Mood to Preferences

"something scary but short to watch with friends tonight"

↓ AI returns JSON:

```
{  
  "genre":    "horror",  
  "timeNeed": "short",  
  "social":   "group",  
  "mood":     "intense"  
}
```

These 5 values become the C engine's input arguments.

*Strict system prompt forces JSON-only output.
Response validated before use — fallback if empty.*

Task 2 — Personalized Explanation

After C engine picks the best title, AI writes why it fits:

"Alien is the perfect pick for a short, intense group night — its claustrophobic tension builds fast and hits hard, exactly the kind of scary that works best when everyone is watching together."

Always references the user's original mood — never generic.

- Same OpenRouter client, same fallback chain
- Rich fallback text if AI fails completely
- Two separate API calls per recommendation

External APIs — TMDB & RAWG

TMDB — The Movie Database

image.tmdb.org · Free API · Hundreds of thousands of films

- Movies page — search by title, 8 genre filters, poster grid
- Movies page — click → cast list, overview, rating, release year
- Discussion page — movie search + backdrop images
- Discussion page — cast with profile photos
- Discussion page — overview sent to AI for analysis
- Poster images fetched at w342 resolution from TMDB CDN

RAWG — Video Games Database

rawg.io · Free API · 800,000+ game titles

- Games page — search by title across 800K+ titles
- Games page — 9 genre filters (Action, RPG, Strategy...)
- Games page — cover art grid with developer & rating
- Games page — click → full description, platforms, tags
- Free tier: 20,000 requests/month (well within our use)

Backend — Express Server & API Endpoints

Method	Endpoint	Purpose	Auth?
POST	/api/recommend	Run full AI + C engine pipeline	—
POST	/api/discuss	AI discussion analysis for a movie	—
GET	/api/titles	Return full 55-title library as JSON	—
POST	/api/auth/register	Create a new user account	—
POST	/api/auth/login	Login and receive a JWT token	—
GET	/api/history	Load user's recommendation history	✓ JWT

DDD Layer: each endpoint is a thin HTTP wrapper · validates input → calls Application layer → returns JSON

No business logic lives in the Interface layer. Business rules stay in Domain & Application.

User Authentication — bcrypt + JWT

Registration Flow



```
// application/auth.js
const hash = await bcrypt.hash(password, 10);
// 10 salt rounds – industry standard
createUser(username, hash);
// Plain password is NEVER stored
```

JSON Web Token (JWT)

- Signed with a secret key on the server
- Contains: user ID + username
- Expires after 7 days automatically
- Frontend stores it in localStorage
- Sent as: Authorization: Bearer <token>
- Protected endpoints verify it on every request

Login Flow



```
const ok = await bcrypt.compare(password, hash);
if (!ok) throw new Error('Wrong credentials');
const token = jwt.sign({id,username}, SECRET,
  { expiresIn: '7d' });
```

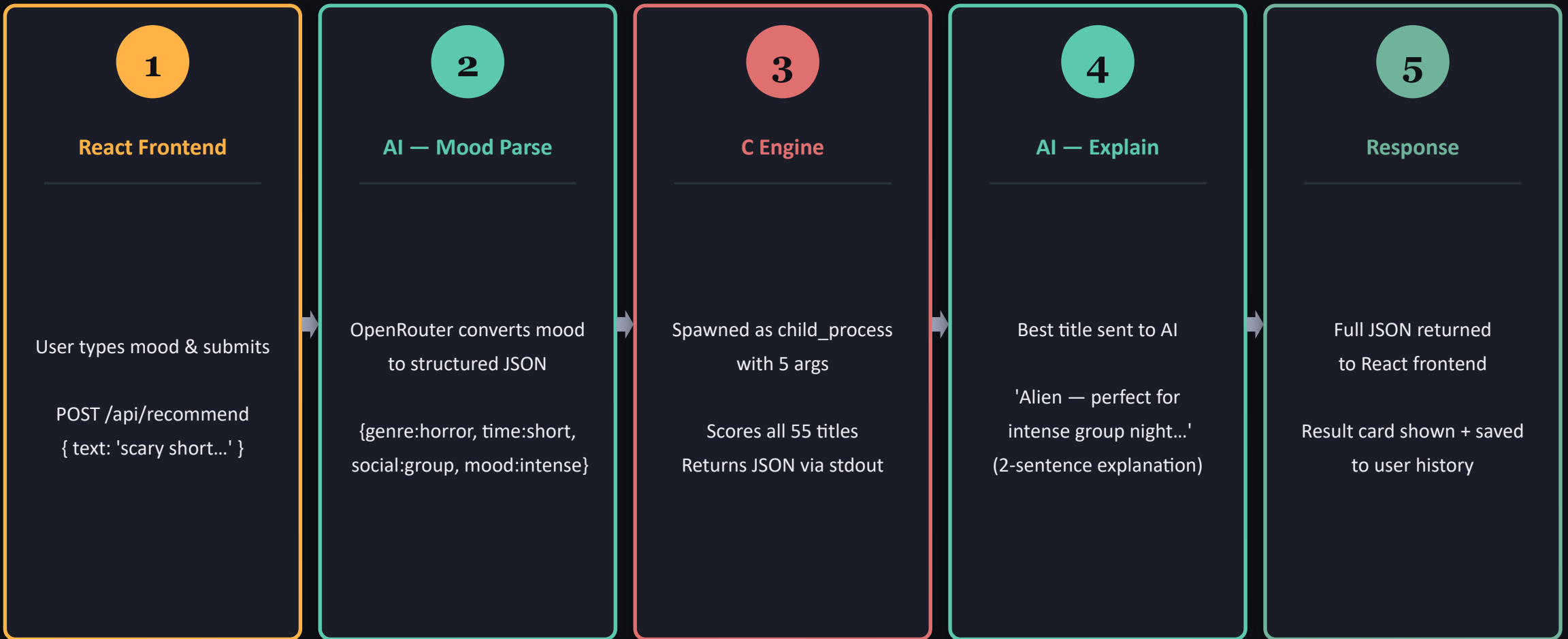
Storage — server/data/users.json



```
{ id, username, passwordHash,
  createdAt, history: [ ...50 entries ] }
// Gitignored – never in GitHub
```

● CONCLUSION

Complete Data Flow — Request Lifecycle



Testing & Evaluation

Mood Input	Expected Genre / Context	Actual Result	Status
"relaxing cozy solo night"	Cozy · Relaxed · Solo	 Balatro (2024)	✓ Pass
"scary horror with a group"	Horror · Intense · Group	 Alien (1979)	✓ Pass
"epic action game for hours"	Action · Long · Solo	 Super Mario Odyssey	✓ Pass
"thoughtful sci-fi, alone"	Sci-Fi · Deep · Solo	 Arrival (2016)	✓ Pass

Edge Cases

Mixed Mood	AI Failure	Empty Input	Gibberish
'funny but also scary' → genre with higher scoring weight wins	All 3 models fail → keyword fallback → still returns a result	Server returns HTTP 400 → frontend shows clear error message	AI still extracts preferences → C engine picks closest match

Limitations & Future Work

Current Limitations

- AI Match recommends only from the 55-title curated library — browse pages use open databases
- App runs locally — not yet deployed on a public server
- Free AI models can be slow or rate-limited during peak hours
- No user feedback loop — the system does not learn from ratings

Future Work

1. User Profiles

Watch history and long-term preferences per account

2. Personalization

Feedback ratings improve future recommendations

3. Larger Library

Expand C engine library from 55 to 500+ titles

4. Cloud Deployment

Host on a server — accessible from any device, anywhere

5. Automated Tests

Unit tests for C scoring logic to prevent regressions

CINEMATCH

Thank You

Questions?

[Member 1]

[Member 2]

[Member 3]

[Member 4]

[Member 5]

[Member 6]